

LMP-GAN: OUT-OF-DISTRIBUTION DETECTION FOR NON-CONTROL DATA
MALWARE ATTACKS

Dissertation

Submitted to

The School of Engineering of the
UNIVERSITY OF DAYTON

In Partial Fulfillment of the Requirements for

The Degree of

Doctor of Philosophy in Engineering

By

David W. Wood

Dayton, Ohio

May, 2024



LMP-GAN: OUT-OF-DISTRIBUTION DETECTION FOR NON-CONTROL DATA
MALWARE ATTACKS

Name: Wood, David W.

APPROVED BY:

Keigo Hirakawa, Ph.D.
Advisory Committee Chairman
Professor, Electrical and Computer
Engineering

Member A. David Kapp, Ph.D.
Committee Member
Researcher, RYWA, United States
Air Force, WPAFB

Member B. Temesgen Kebede, Ph.D.
Committee Member
Professor, Electrical and Computer
Engineering

Member C. Raúl Ordóñez, Ph.D.
Committee Member
Professor, Electrical and Computer
Engineering

© Copyright by
David W. Wood
All rights reserved
2024

ABSTRACT

LMP-GAN: OUT-OF-DISTRIBUTION DETECTION FOR NON-CONTROL DATA MALWARE ATTACKS

Name: Wood, David W.
University of Dayton

Advisor: Dr. Keigo Hirakawa

Anomaly detection is a common application of machine learning. Out-of-distribution (OOD) detection in particular is a semi-supervised anomaly detection technique where the detection method is trained only on the inlier (in-distribution) samples—unlike the fully supervised variant, the distribution of the outlier samples are never explicitly modeled in OOD detection tasks. In this work, we design a novel GAN-based OOD detection network specifically designed to protect a cyber-physical signal systems from novel Trojan malware called non-control data (NCD) attack that evades conventional malware detection techniques. Inspired in part by the classical locally most powerful (LMP) test in statistical inferences, the proposed LMP-GAN trains the OOD detector (discriminator) by generating OOD samples that are aimed at making maximal alteration to the inlier samples while evading detection. We experimentally compare the results to the state-of-the-art anomaly detection methods to demonstrate the benefits and the appropriateness of the LMP-GAN OOD detector.

The pursuit of authenticity is not a straight line and seldom has a clear path. Rather, it is defined by choices to sacrifice what is easy for what is true.

It is not imposition or conquest over others, but dominance over ourselves - our own fears, internalized criticisms, and self-doubt. The further we deviate from convention, the greater resistance we will find in the everyday.

But in that fight for identity and expression comes freedom and fulfillment. **Let your true self shine and accept that others may need sunglasses to see you.** In time, they will adapt or avert their gaze.

And you will thrive.

Shine, my friends. - Adam McCash

Dedicated to and in loving memory of Sir Adam Reid-McCash

May 29th, 1983 - May 31, 2019

ACKNOWLEDGMENTS

Family:

I would first like to acknowledge my chosen family. I have climbed and moved mountains in my life, only because I've had them to hold me up and support me when I fall.

Patrick Dull, Luis Suarez, Jephthah Lawson, Eli Johnson, Thomas Burske, Tracy Burske, Kurt Jacobs, Laura Goldsmith, David Gonzolas, and Bryan Emerson.

Faculty, Lab & Committee:

To the most amazing advisor, and mentor: Dr. Keigo Hirakawa

It feels like fate, that I had waited till now to partake in my PhD., as I am unsure if I would have been successful with anyone else. You have been supportive, patient, kind and understanding since the beginning. But the most important to me, you always wanted me to be myself and never once asked me to compromise that.

Thank you to the support of Dr. Raul Ordonez, Dr. David Kapp, and Dr. Temesgen Kebede

To the amazing support of the friends I made in the ISSL during my time at UD. They helped me through difficult medical times, and celebrated with me during out tremendous successes.

Sponsored by WPAFB through DAGSI:Defense Associated Graduate Student Innovators

TABLE OF CONTENTS

ABSTRACT	3
DEDICATION	4
ACKNOWLEDGMENTS	5
LIST OF FIGURES	8
LIST OF TABLES	10
LIST OF ACRONYMS	11
CHAPTER I. INTRODUCTION	12
CHAPTER II. BACKGROUND AND REVIEW	14
2.1 Non-Control Data Attacks	14
2.2 Existing Mitigation Strategies	16
CHAPTER III. NCD-ATTACKS	17
3.1 Analysis of NCD Attack on Signal Systems	17
3.2 NCD Attack Detection As Minimax Problem	20
CHAPTER IV. PROPOSED: LMP-GAN OOD DETECTION	24
4.1 Proposed: LMP-GAN	24
4.2 LMP-GAN Implementation	26
CHAPTER V. EXPERIMENTAL EVALUATION	29
CHAPTER VI. SYNTHETIC OOD DETECTION EXPERIMENTS	30
CHAPTER VII. APPLIED DATASETS AND RESULTS	34
7.1 Malware Dataset	34
7.2 Credit Card Fraud Dataset	35
CHAPTER VIII. GAN ABLATION STUDY	37
CHAPTER IX. MODEL ARCHITECTURE AND TUNING	39
CHAPTER X. FUTURE AND RELATED WORK	47

10.1	Future Work	47
10.1.1	Generative Models	47
10.1.2	Anomaly Detection	48
10.2	Related Work	50
CHAPTER XI. CONCLUSION		53
BIBLIOGRAPHY		55

LIST OF FIGURES

2.1	Control flow of normal and compromised signal/data processing systems. Blue and orange colors represent normal and compromised components, respectively. (a) Normal systems operate on the input data to yield output. (b) In a conventional Trojan, the trigger alters the behavior of the software by causing a branch or jump instruction. (c) In NCD attack, an attack is always being carried out, although the impact on the output is benign (by design) except in some specific cases.	15
3.1	Binary classification task performed on multi-dimensional multi-class malware data (showing a two dimensional slice). (a-b) The result of supervised learning, trained on the normal (θ_0) and known malware samples (θ_1). (c-e) The result of semi-supervised learning, trained on normal samples (θ_0) only. (a) Supervised detector applied to raw data to detect some known malwares (θ_1). (b) Through training, detector's feature extraction improves the detection of known malwares (θ_1) by learning to transform data to better separate normal (θ_0) and known malware (θ_1) samples. However, the detection of novel malware (θ_2) is still poor. (c) The generator in GAN designed for domain expansion learns the distribution of the inlier data, which is used as a proxy outlier in OOD detection work in [1–5]. Although the classifier tightens the boundary around the inlier samples (to exclude the as many generator samples), there exists no feature extraction technique that would separate the inlier samples from the nearly identically distributed generative samples. (d-e) Adversarial generator in the proposed LMP-GAN learns to yield samples near the discriminator's boundary, mimicking NCD attack. Thus the discriminator has the incentive to tighten the classification boundary, as well as to learn the feature extraction to better cluster the inlier samples until it is no longer possible to condense the inlier set.	18
3.2	Illustration of proof of Theorem 1. The extremum $\Gamma_{\hat{\theta}}(X)$ must lie in the convex set Ω_{τ} as well as on the boundary of Υ_{κ} . Since $\Omega_{\tau} \cap \Upsilon_{\kappa} = \Omega_{\tau}$, $\Gamma_{\hat{\theta}}(X)$ must lie on the boundary of Ω_{τ}	23
4.1	Comparison of GAN structures. (a) In conventional GANs used in domain expansion, the generator learns to mimic the system behavior. The generator loss function is aimed at minimizing $\delta(X, \Phi_{\lambda})$. (b) In LMP-GAN, the generator learns to draw samples on the discriminator's boundary ($\delta \approx \tau$), mimicking NCD attacks. The discriminator is a binary classifier learning to predict the inlier/outlier label. By incentivising the discriminator to reduce the cardinality of the boundary set, the discriminator learns to cluster the inliers compactly.	26

6.1	Out-of-distribution detection results comparing Neyman-Pearson (NP solid lines, theoretical bound performance) [6], multilayer perceptron (MLP, dash-dot lines, supervised) [7], and the proposed LMP-GAN (dashed lines, semi-supervised). (a) The inliers and outliers are N dimensional Gaussian and Laplacian vectors, respectively. (b) The inlier and outliers are independent and correlated Gaussian vectors, respectively. (c) Result of applying network trained to discriminate H_0 and H_1 in (6.4), tested on H_0 v.s. H_2 . Despite being trained only on inlier samples, it is competitive with the theoretically optimal binary classification result and outperforms MLP trained with inlier and outlier samples.	31
9.1	Plot of 9.4 results of each architectural change done to the LMP-GAN and the impact on ROC and detection/False-Positive rate. (a) shows the standard ROC-AUC where some models clearly underperform for the task, like "Gen Unet-5" and "Lean Full Conv." The architecture changes are much closer in the 0 to 0.1 False positive range. (b) By applying the log scale to the X-axis, the performance of the various models, at .01 and .001, becomes easier to notice. "Full-Conv" being the second best when compared to the Proposed model.	45

LIST OF TABLES

4.1	Sequential Model implemented in Tensorflow for LMP-GAN Discriminator with sigmoid output.	28
4.2	Sequential Model implemented in Tensorflow for LMP-GAN Generator with output matching system sample size.	28
6.1	The out-of-distribution (OOD) detection results tested on synthetic data. Hypotheses tests in (6.1) and (6.2) are carried out on vectors of lengths $N=10/N=30/N=50$; hypothesis test in (6.4) is conducted for $N=50$. Neyman-Pearson represents the theoretical limit to each binary classification task.	31
7.1	The detection performance of LMP-GAN and the state-of-the-art alternatives tested on Malware API Dataset in [8].	35
7.2	The detection performance of LMP-GAN and the state-of-the-art alternatives tested on Credit Card Fraud Dataset in [9].	36
8.1	Ablation study examining specific design choices that make LMP-GAN unique and effective. We compared the GAN network structures (Figure 4.1(a) vs Figure 4.1(b)), generator loss functions (minimax $\tau = 0.5$ vs domain expansion $\tau = 0$), and outlier generations (neural network generator vs noise). All methods trained on length $N=50$ vectors for 2350 epochs.	37
9.1	Sequential Model implemented in TensorFlow for LMP-GAN Generator with sigmoid output using a U-Net architecture with 1-Dimensional Convolution. . . .	41
9.2	Sequential Model implemented in TensorFlow for LMP-GAN Generator with sigmoid output using a "Lean" UNet architecture with 1-Dimensional Convolution. Lean is defined as removing the Dense post layers from the Conv1D U-net Architecture.	42
9.3	Sequential Model implemented in Tensorflow for LMP-GAN Discriminator with sigmoid output using an Encoder architecture with 1-Dimensional Convolution. .	43
9.4	Architecture changes to the proposed GAN by modification of the generator and discriminator using Convolutional Neural networks. Each method tests a structural change using 6.2 to test the effects of CNNs on LMP-GAN performance. False Positive (FP) rates at .01 and .001 are considered for each model as a testing parameter, given that the model's effectiveness is based on novel malware detection.	44

LIST OF ACRONYMS

- AUC - Area Under Curve
- AWGN - Additive White Gaussian Noise
- DGCNN - Deep Graph Convolutional Neural Network
- GAN - Generative Adversarial Network
- LMP - Locally Most Powerful
- MLP - Multi-Layer Perceptron
- MOE - Mixture of Experts
- NCD - Non-Control Data
- NP - Neyman-Pearson
- OOD - Out-of-distribution
- PCA - Principle component Analysis
- ROC - Reciever Opperator Curve

CHAPTER I

INTRODUCTION

Cyber security threats grow with the modern society's dependence on technological advances. Every electronic component and system in use is at risk of malware attacks. Some attacks are aimed at exposing or stealing valuable information. Other types of attacks alter the system's behavior to make them inoperable or to falsify the system's output. This paper is concerned with attacks on signal systems designed to manipulate the input-output relationship of a signal/data processing unit to cause harm. Examples may include altering sensor data (GPS, speedometer, etc.) or control signals (robotics, autonomous systems, etc.) to manipulate or impair the cyber-physical systems that depend on them. *Stuxnet* is one such example that caused substantial damage in 2005 to the centrifuges of Iranian nuclear programs by causing it to spin irregularly [10]. Vehicles today are equipped with unprecedented number of sensors that, while enhancing safety under normal operations, also increase the risks to system faults and exploitation's [11].

To protect against such vulnerabilities, detection of Trojan attacks has been an active area of research. In a seminal article by Chen *et al.* [12] and later elaborated by many [13–19], a novel malware threat called *non-control data* (NCD) attack for signal/data systems is theorized. NCD attack refers to a set of deeply embedded malicious instructions designed to alter the program behaviors without explicit triggers, which would evade detection by the conventional malware detection techniques looking for evidences of branching triggers [14, 20, 21]. Although NCD attacks on signal processing units have not yet been reported in the wild (to the best of our knowledge), we develop detection strategies to protect against this potential threat.

We propose a semi-supervised anomaly detection method based on novel out-of-distribution (OOD) modeling to protect signal systems against NCD attacks. The goal is to design a data-driven binary classification algorithm that learns to discriminate the correct signal system input-output relationship from an abnormal one. In order to be robust to previously unseen attacks, we limit the training to the observable signal corresponding to normal system behavior only (i.e. without the explicitly defined abnormal training data). We achieve this by proposing a novel generative adversarial network (GAN) training strategy called LMP-GAN modeled after locally most powerful hypothesis test.

The remainder of the paper is organized as follows. In Section II we provide a requisite background on the NDC malware attacks and the existing techniques for detecting them. The OOD detection-based anomaly detection task for NCD attack on signal system is formalized as a minimax optimization problem in Section IV, giving rise to the proposed LMP-GAN solution. In Section V we present the experimental results to verify superior performance of LMP-GAN as OOD detection capable of handling complex real-world data, before making concluding remarks in Section XI.

CHAPTER II

BACKGROUND AND REVIEW

2.1 Non-Control Data Attacks

The conventional wisdom in Trojan detection is to detect the change point as it occurs. As shown by Figure 2.1(b), the trigger may alter the behavior of the software by causing a branch or jump instruction in the program, causing the computing hardware to begin executing a different instruction sequence to deviate from its default behavior. Known as program *control flow*, the state-of-the-art methods monitor the order in which instructions are executed to detect deviations from the usual pattern of control transfers [20–26] or impose restriction to the control plane memory accesses [27, 28]. Since the system can be compromised whenever a branch instruction can be forced (control flow *hijacking*), typical malicious code does not need to be embedded within the executable code [14, 20, 21].

What if the program control flow was never altered by the Trojan? A number of recent publications report the feasibility and the potential threats from a novel class of Trojans called non-control data (NCD) or *data only* attack that can be carried out without an explicitly defined trigger [12–19]. Specifically, the malware averts detection by employing a *soft* trigger, altering the decision-critical data/signal without explicitly executing branching or jump instructions. The instruction set used to carry out the software’s normal behavior already includes malicious code that *always executes* with negligible/benign behavioral impact during normal operation. However, the same malicious code will exert a greater influence over the program output near the deterministic trigger point to carry out the attack without altering the control flow or the memory access patterns. For this reason,

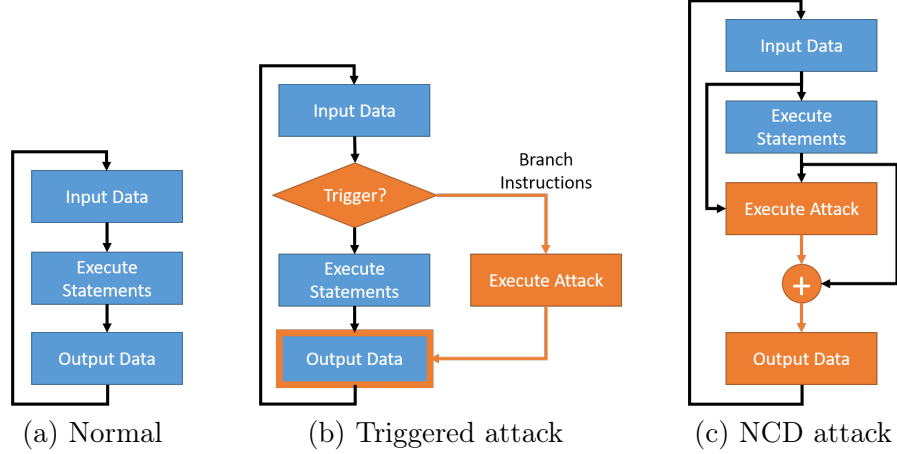


Figure 2.1: Control flow of normal and compromised signal/data processing systems. Blue and orange colors represent normal and compromised components, respectively. (a) Normal systems operate on the input data to yield output. (b) In a conventional Trojan, the trigger alters the behavior of the software by causing a branch or jump instruction. (c) In NCD attack, an attack is always being carried out, although the impact on the output is benign (by design) except in some specific cases.

algorithms designed to detect trigger point by control flow or memory access will not be able to distinguish between benign and compromised behavior. See Figure 2.1(c).

NCD attack vulnerabilities have been theorized and/or demonstrated in server privilege escalation [16], untrusted code import in browsers [17,19,29], leakage of secret keys [18], and memory corruption [21]. Recent efforts to protect against NCD attacks involve memory/data flow provenance and protection [30,31], CPU register protection [32], pointer taintedness [33], and data hiding [34]. Since one must gain access to the executable to embed malicious code as illustrated in Figure 2.1(c), the NCD attack would likely be introduced by someone (e.g. in the supply chain) with a greater access to the source code rather than by the end user who would download the malicious code (more common in control flow hijacking). Nevertheless, recent advances in the programming techniques makes NCD attacks on signal systems easier than ever to carry out [15,16,35,36].

2.2 Existing Mitigation Strategies

Since NCD attacks cannot be detected by monitoring control flow changes, data-based anomaly detection is considered to be among the critical defensive cyber security techniques. Formally, program anomaly detection is carried out by detecting the deviation from modeled/learned correct system behavior [37–40]. Existing anomaly detection specific to NCD attacks rely on application-, program-, or computing hardware-specific attributes, such as pointer de-referencing [33], analysis of system call *misuses* [41,42], dataflow [43] and timing requirements [44].

Yet, there still seems to be a gap in the literature on anomaly detection designed for NCD attacks on signal/data processing programs and devices. A threat posed by NCD-type attack is that the input-output relationship in mission-critical signal systems such as measurement devices, sensors, control units, navigational guidance units, etc. can be altered to cause harm on the cyber-physical systems that rely on them. A particularly challenging aspect to detecting anomalies in the input-output signal relations is the stochastic nature of data/signal that is being processed, since real-world signal processing units comprised of a series of complex computational operations are expected to handle noisy inputs. (Contrast this to anomaly detections on system call [41,42] and pointer de-referencing [33] that are exact in nature.) For example, an NCD attack falling under the noise floor may be undetectable, although the extent of harm may be limited also (i.e. *benign*). Conversely, large noise values can be falsely detected as system anomaly, making it difficult for malware detector to operate in a low false alarm regime. Developed in the remainder of this paper is a principled approach to data-driven out-of-distribution detection task designed to minimize the harm caused by NCD attacks (instead of the detection accuracy that state-of-the-art anomaly detection methods are optimized for [45–47]).

CHAPTER III

NCD-ATTACKS

3.1 Analysis of NCD Attack on Signal Systems

Anomaly detection is considered an important layer of defense against an NCD attack that cannot be detected by monitoring control flow changes [33, 41–44]. In this work, we propose a machine learning-based detection algorithm designed to detect anomalies in the input-output signal relations. Furthermore, to protect against unseen/unlabeled and deeply embedded malware, we focus on a particular flavor of anomaly detection task called out-of-distribution (OOD) detection—a *semi-supervised* setup where the training data is comprised entirely of the *inlier* class representing the normal system behavior (i.e. no training data of the *outliers* class).

Formalizing OOD, let $\Gamma_\theta : \mathbb{R}^M \rightarrow \mathbb{R}^N$ denote the system that generates output signal $Y = \Gamma_\theta(X) \in \mathbb{R}^N$ based on input signal $X \in \mathbb{R}^M$ (not necessarily deterministic). Let $\theta \in \Theta$ represent the multivariate parameter in admissible parameter space Θ describing the system state, with $\theta_0 \in \Theta$ describing the correct behavior of the system. Thus we consider the following hypothesis test:

$$\begin{cases} H_0 : \theta = \theta_0 \text{ (inlier/normal)} \\ H_1 : \theta \neq \theta_0 \text{ (outlier/anomaly/compromised).} \end{cases} \quad (3.1)$$

The corresponding likelihood function $f : \mathbb{R}^M \times \mathbb{R}^N \rightarrow \mathbb{R}$, $f(Y|X, \theta)$ describes the probability density function of the output $Y = \Gamma_\theta(X)$ conditioned on the input X and the system parameter θ .

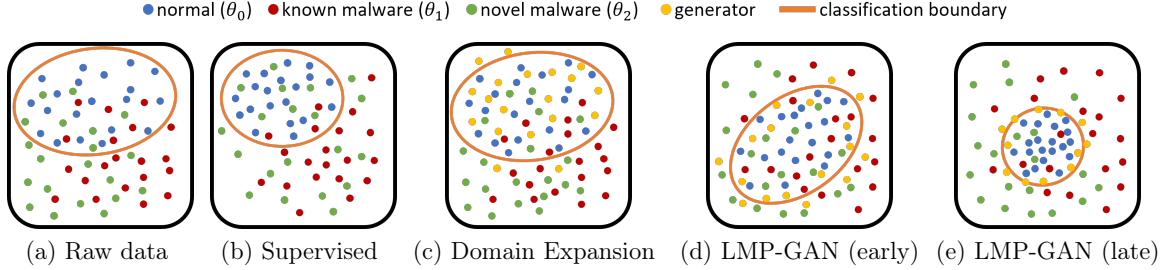


Figure 3.1: Binary classification task performed on multi-dimensional multi-class malware data (showing a two dimensional slice). (a-b) The result of supervised learning, trained on the normal (θ_0) and known malware samples (θ_1). (c-e) The result of semi-supervised learning, trained on normal samples (θ_0) only. (a) Supervised detector applied to raw data to detect some known malwares (θ_1). (b) Through training, detector’s feature extraction improves the detection of known malwares (θ_1) by learning to transform data to better separate normal (θ_0) and known malware (θ_1) samples. However, the detection of novel malware (θ_2) is still poor. (c) The generator in GAN designed for domain expansion learns the distribution of the inlier data, which is used as a proxy outlier in OOD detection work in [1–5]. Although the classifier tightens the boundary around the inlier samples (to exclude the as many generator samples), there exists no feature extraction technique that would separate the inlier samples from the nearly identically distributed generative samples. (d-e) Adversarial generator in the proposed LMP-GAN learns to yield samples near the discriminator’s boundary, mimicking NCD attack. Thus the discriminator has the incentive to tighten the classification boundary, as well as to learn the feature extraction to better cluster the inlier samples until it is no longer possible to condense the inlier set.

Under this formalism, the goal is to design a data-driven binary classification algorithm $\delta : \mathbb{R}^M \times \mathbb{R}^N \rightarrow \mathbb{R}$ that learns to discriminate the correct signal system behavior $f(Y|X, \theta = \theta_0)$ (the null hypothesis H_0) from an abnormal one $f(Y|X, \theta \neq \theta_0)$ (the alternative H_1). Let $\delta(X, Y) \in \mathbb{R}$ denote the score of malware detection, where the input-output pair (X, Y) whose detection score exceeds the detection threshold τ (i.e. $\delta(X, Y) > \tau$) is determined to be malware/outlier. A number of solutions for binary classification exists in the literature of statistical inferences. For instance, the Neyman-Pearson is a *provably optimal* binary classifier that leverages a likelihood ratio test of the form [6, 48, 49]:

$$\delta(X, Y) = \frac{f(Y|X, \theta = \theta_1)}{f(Y|X, \theta = \theta_0)}. \quad (3.2)$$

Neyman-Pearson is also analogous to the supervised machine learning model illustrated in Figure 3.1(a) where the alternative θ_1 or its corresponding likelihood $f(Y|X, \theta = \theta_1)$ must either be known *a priori* or be deduced from training data [50]. Feature extraction learned through contrastive training transforms the data (X, Y) in a manner that best separates $Y|X, \theta_0$ and $Y|X, \theta_1$ to further improve the classification accuracy. As illustrated in Figure 3.1(b), however, the binary classifier of the form (3.2) will fail unsurprisingly to properly classify $Y|X, \theta_2$ corresponding to a parameter θ_2 that it was not trained/designed for. In the cybersecurity context, $f(Y|X, \theta_1)$ is a distribution of *known* threats that may be deduced from existing samples, whereas $f(Y|X, \theta_2)$ represents a *novel* malware that is previously unencountered.

In OOD, neither the H_1 likelihood function $f(Y|X, \theta \neq \theta_0)$ nor the prior density function $f(\theta)$ are assumed to be known because the training data is limited to the correctly signal system behavior $f(Y|X, \theta = \theta_0)$ [47]. When θ_1 is generally not known but the likelihood

function is differentiable, we typically use the locally most powerful (LMP) tests [49, 51, 52] or locally most mean powerful unbiased (LMMPU) tests [52, 53]—detectors sensitive to the most difficult case $\theta_1 \approx \theta_0$ (i.e. the alternative is similar but not identical to θ_0). However, LMP and LMMPU tests cannot be implemented in a data-driven manner because they threshold the derivative $\frac{\partial}{\partial \theta} \ln f(Y|X, \theta)$ that cannot be learned from observing samples drawn from the null hypothesis $f(Y|X, \theta = \theta_0)$ only [49, 52, 53].

There are NCD attack-specific cyber security considerations for the signal systems, also. The *extent of the harm* caused by the output signal $Y = \Gamma_\theta(X)$ from a compromised system $\theta \neq \theta_0$ is assumed to be measured by the deviation $\Delta = \Gamma_\theta(X) - \Gamma_{\theta_0}(X)$ from the correct output $\Gamma_{\theta_0}(X)$. In this sense, an attack on the signal system is said to be *benign* (e.g. below noise floor) if Δ is small. Thus the adversarial objective of the NCD attacker is

$$\max_{\theta: \delta(X, \Gamma_\theta(X)) \leq \tau} \|\Gamma_\theta(X) - \Gamma_{\theta_0}(X)\| \quad (3.3)$$

for some detection threshold τ and norm $\|\cdot\|$. That is, the attacker seeks to maximize the harm ($\max \|\Delta\|$) while still evading detection ($\delta(X, \Gamma_\theta(X)) \leq \tau$).

As a side note, the norm function $\|\cdot\|$ that quantifies the harm caused by NCD attack cannot always be defined explicitly (although there may be applications where the harm can be quantified). Fortunately, Theorem 1 below can be proven for any norm function, and thus the remainder of this paper is developed in a norm-agnostic manner.

3.2 NCD Attack Detection As Minimax Problem

In this work, an OOD detection algorithm based on generative adversarial network (GAN) is proposed. The idea to use GAN in OOD detection task itself is not new [1–5]. Known as *domain expansion*, a generator $Y = \Phi_\lambda(X, N)$ with parameters λ may be trained

to mimic the distribution of a limited H_0 sample set:

$$\Phi_\lambda(X, N) \sim f(Y|X, \theta = \theta_0), \quad (3.4)$$

where $N \in \mathbb{R}^P$ is a seed noise. As illustrated in Figure 4.1(a), state-of-the-art OOD detectors have used $\Phi_\lambda(X, N)$ as a proxy for outliers H_1 in (3.2)—a binary classifier δ is trained to discriminate the generated samples $\Phi_\lambda(X, N)$ from the real samples $\Gamma_{\theta_0}(X)$. Domain expansion-based OOD detection is known to have a high risk of type 1 errors, however, because one cannot train feature extraction to separate $\Phi_\lambda(X, N)$ and $\Gamma_{\theta_0}(X)$ that are indistinguishable in distribution (by design) [3, 4, 54], as depicted in Figure 3.1(c). In addition, the *domain expansion* is a poor proxy for the NCD attack in (3.3) since (3.4) implies that H_1 is benign (generator learns to mimic harmless system behavior H_0).

Recalling (3.3), we instead aim to solve the following minimax problem:

$$\min_{\delta} \max_{\theta: \delta(X, \Gamma_\theta(X)) \leq \tau} \|\Gamma_\theta(X) - \Gamma_{\theta_0}(X)\| \quad (3.5)$$

for some the threshold τ set to yield a target false alarm rate α (following the usual practices of Neyman-Pearson setup). In other words, OOD detection algorithm δ seeks to minimize the harm that NCD attack may cause. Consider the following theorem describing the *topology* of the NCD attacks.

Theorem 1 *Let δ be any detector whose the set of inliers $\Omega_\tau = \{Y : \delta(X, Y) \leq \tau\}$ is a convex subset of the range space of $\Gamma_\theta(X)$ containing $Y = \Gamma_{\theta_0}(X)$. Define $\hat{\theta} \in \Theta$ to be the maximizer of (3.3):*

$$\hat{\theta} = \arg \max_{\theta: \delta(X, \Gamma_\theta(X)) \leq \tau} \|\Gamma_\theta(X) - \Gamma_{\theta_0}(X)\|. \quad (3.6)$$

Then $\delta(X, \Gamma_{\hat{\theta}}(X)) = \tau$.

Proof: The hypothesis of the theorem allows us to simplify the NCD objective function in (3.3) as

$$\max_{Y \in \Omega_\tau} \|Y - \Gamma_{\theta_0}(X)\|. \quad (3.7)$$

Define $\kappa = \|\Gamma_{\hat{\theta}}(X) - \Gamma_{\theta_0}(X)\|$ to be the extremal *harm* caused on the signal/data processing system. Owing to the fact that $\|\cdot\|$ is a norm,

$$\Upsilon_\kappa = \{Y : \|Y - \Gamma_{\theta_0}(X)\| \leq \kappa\} \quad (3.8)$$

is a convex set. The intersection $\Omega_\tau \cap \Upsilon_\kappa$ of two convex sets must also be convex, and its boundary must be defined by the boundaries of Ω_τ and Υ_κ . Moreover, owing to the fact that no point in Ω_τ must lie outside of Υ (a restriction imposed by (3.7)), the intersection reduces to $\Omega_\tau \cap \Upsilon_\kappa = \Omega_\tau$. But at least one point in Ω_τ must lie on the boundary of Υ_κ , thus the extremum point $\hat{Y} = \Gamma_{\hat{\theta}}(X)$ must lie at the boundary of Ω_τ . See also the illustration in Figure 3.2. $\square$

We conclude from Theorem 1 that the NCD attack causes maximum harm $\hat{Y} = \Gamma_{\hat{\theta}}(X)$ at some point on the boundary $\frac{\partial}{\partial \tau} \Omega$ of the inlier set Ω_τ ,

$$\hat{Y} \in \frac{\partial}{\partial \tau} \Omega_\tau = \{Y : \delta(X, Y) = \tau\}. \quad (3.9)$$

Thus the minimax optimization problem in (3.5) is equivalent to another minimax task:

$$\min_{\delta} \max_{Y \in \frac{\partial}{\partial \tau} \Omega_\tau} \|Y - \Gamma_{\theta_0}(X)\|. \quad (3.10)$$

As we shall see in the next section, the minimax optimization task in (3.10) naturally lends itself to generative adversarial network framework.¹

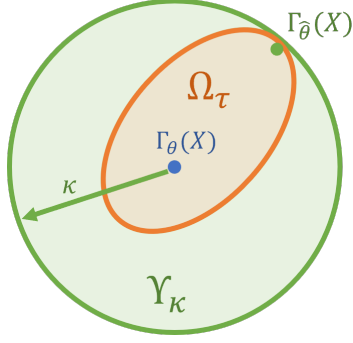


Figure 3.2: Illustration of proof of Theorem 1. The extremum $\Gamma_{\hat{\theta}}(X)$ must lie in the convex set Ω_τ as well as on the boundary of Υ_κ . Since $\Omega_\tau \cap \Upsilon_\kappa = \Omega_\tau$, $\Gamma_{\hat{\theta}}(X)$ must lie on the boundary of Ω_τ .

¹Conventional GAN is often regarded as a minimax solver, in the sense that the *generator* aims to minimize the discriminator detection score. We contrast this to the proposed minimax problem in (3.5) where the *discriminator* is tasked to minimize the maximum deviation (harm) caused by the generator (NCD attack). The difference between the two distinct minimax problems is examined in ablation study.

CHAPTER IV

PROPOSED: LMP-GAN OOD DETECTION

4.1 Proposed: LMP-GAN

The challenge to directly solving (3.5) is that *we do not know* the precise values of the maximally harmful output $\hat{Y} = \Gamma_{\hat{\theta}}(X)$ and the ideal system output $\Gamma_{\theta_0}(X)$. Motivated by Theorem 1 and (3.10), however, we postulate that a discriminator δ can be improved further by regarding the boundary points $\frac{\partial}{\partial \tau} \Omega_\tau$ containing the adversarial NCD attack point $\hat{Y} = \Gamma_{\hat{\theta}}(X)$ as proxy for H_1 . That is, we propose learn to discriminate between the H_0 training data drawn from $f(Y|X, \theta = \theta_0)$ and proxy H_1 samples drawn from $\frac{\partial}{\partial \tau} \Omega_\tau$. It is clear from Figure 3.2 that the minimax objective in (3.5) can be met iteratively by shrinking the size of Ω_τ , which has the overall effect of shrinking $\Omega_\tau \cap \Upsilon_\kappa$.

Thus we propose to carry out a generative adversarial training as illustrated in Figure 3.1(d). Let $\mathcal{I} = \{(X_1, Y_1), \dots, (X_K, Y_K)\}$ denote K training H_0 samples. Consider an adversarial loss function that helps the generator $\Phi_\lambda : \mathbb{R}^M \times \mathbb{R}^N \times \mathbb{R}^p \rightarrow \mathbb{R}^N$ learn to draw samples from the boundary set $\frac{\partial}{\partial \tau} \Omega_\tau = \{Y : \delta_\eta(X, Y) = \tau\}$ of a given detector $\delta_\eta : \mathbb{R}^M \times \mathbb{R}^N \rightarrow [0, 1]$ operating at the threshold $\tau \in (0, 1)$, as follows:

$$\mathcal{L}_\Phi(\lambda) = \frac{1}{K} \sum_{k=1}^K \left| \delta_\eta(X_k, \Phi_\lambda(X_k, Y_k, N_k)) - \tau \right|, \quad (4.1)$$

where $N_k \in \mathbb{R}^p$ is the k th instantiation of random seed. This loss function discourages the generator that are less harmful $\delta_\eta(X, \Phi_\lambda(X, Y, N)) < \tau$. It also penalizes the generator that fails to evade detection $\delta_\eta(X, \Phi_\lambda(X, Y, N)) > \tau$. (As a side note, conventional generators are designed to evade detection. This is akin to minimizing the generator loss in (4.1) with τ set to 0—least likely to be detected but NCD attacks are benign.)

On the other hand, the discriminator performance is evaluated based on the binary cross-entropy loss to measure the difference between the training set $\mathcal{I}$ and the boundary set $\frac{\partial}{\partial \tau} \Omega_\tau$ containing the adversarial NCD attack point $\hat{Y} = \Gamma_{\hat{\theta}}(X)$, in the following sense:

$$\mathcal{L}_\delta(\eta) = \frac{1}{K} \sum_{k=1}^K \ln \delta_\eta(X_k, Y_k) + \frac{1}{K} \sum_{k=1}^K \ln(1 - \delta_\eta(X_k, \Phi_\lambda(X_k, Y_k, N_k))). \quad (4.2)$$

The generative and discriminative models train iteratively, minimizing their respective loss functions as follows:

$$\hat{\lambda} = \arg \min_{\lambda} \mathcal{L}_{\Phi(\lambda)}$$

$$\hat{\eta} = \arg \min_{\eta} \mathcal{L}_\delta(\eta) \quad (4.3)$$

Early in the training process, the entropy between the training set $\mathcal{I}$ and the generative set $\{\dots, (X_k, \Phi_\lambda(X_k, Y_k, N_k)), \dots\}$ decreases as the number of outliers sampled by the generator will fall (Figure 3.1(d)). It is countered by the discriminator training that works to reduce the set of feasible inliers Ω_τ to exclude as much of boundary set $\frac{\partial}{\partial \tau} \Omega_\tau$ without worsening type 1 errors. Since this adversarial training incentivizes the discriminator's feature extraction to learn to reduce the *cardinality* of the convex boundary set $\frac{\partial}{\partial \tau} \Omega_\tau$, the cardinality of Ω_τ is also expected to reduce (i.e. inliers are clustered), as illustrated in Figure 3.1(e).

As an additional justification, one may interpret the proposed GAN model as a machine learning-variant of LMP or LMMPU-based OOD detection. Denote by $\Omega'_\tau = \{Y : \delta(X, Y) \leq \tau\}$ the inlier set corresponding to the classical likelihood ratio test in (3.2). When $\theta_1 \approx \theta_0$

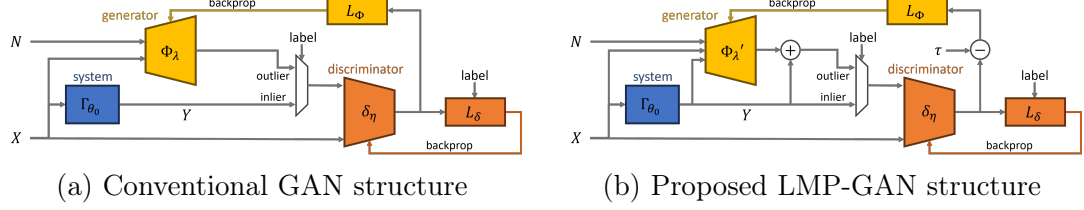


Figure 4.1: Comparison of GAN structures. (a) In conventional GANs used in domain expansion, the generator learns to mimic the system behavior. The generator loss function is aimed at minimizing $\delta(X, \Phi_\lambda)$. (b) In LMP-GAN, the generator learns to draw samples on the discriminator’s boundary ($\delta \approx \tau$), mimicking NCD attacks. The discriminator is a binary classifier learning to predict the inlier/outlier label. By incentivising the discriminator to reduce the cardinality of the boundary set, the discriminator learns to cluster the inliers compactly.

(i.e. θ_1 similar but not identical to θ_0), the *boundary* $\frac{\partial}{\partial \tau} \Omega'_\tau$ can be written as

$$\frac{\partial}{\partial \tau} \Omega'_\tau = \left\{ Y : \frac{\partial}{\partial \theta} \ln f(Y|X, \theta) = \tau' \right\}, \quad (4.4)$$

where $\tau' = \frac{\tau-1}{\theta_1-\theta_0}$. Recalling that LMP-type detectors perform inference by thresholding $\frac{\partial}{\partial \theta} \ln f(Y|X, \theta)$, the LMP test is an analytical solution to finding the boundary set $\frac{\partial}{\partial \tau} \Omega'_\tau$ that minimizes the cardinality of Ω'_τ without worsening type 1 errors (as does our method in (4.1)-(4.3)). For this reason, we refer to the proposed GAN-based solution to (3.10) in (4.1)-(4.3) as the *LMP-GAN*.

4.2 LMP-GAN Implementation

The overall goal of LMP-GAN is to train the discriminator $\delta_\eta(X, Y)$ solve minimax problems in (3.5) and (3.10). The high level structure for the proposed LMP-GAN is shown in Figure 4.1(b). We describe each of the components below.

The discriminator is implemented as a 6-layer neural network. Each layer is fully connected, with a scaled size based on dimensional scaling of a set value to increase layer size

by ratio while maintaining structural shape. The structure of the model can be seen in Table 4.1 with total parameters for an input of size 50, and structural dimensions scaled from 1000.

For the design of generator in LMP-GAN, we draw inspirations from residual neural network (ResNet) [55]. Suppose we rewrite (3.10) in terms of the residual term $\Delta = Y - \Gamma_{\theta_0}(X)$:

$$\min_{\delta} \max_{\Delta: \Delta + \Gamma_{\theta_0} \in \frac{\partial}{\partial \tau} \Omega_{\tau}} \|\Delta\|. \quad (4.5)$$

Equivalently, we implement the generator in LMP-GAN as a residual neural-network training for the input-output training pair (X_k, Y_k) :

$$\Phi_{\lambda}(X_k, Y_k, N_k) = Y_k + \Phi'_{\lambda}(X_k, Y_k, N_k), \quad (4.6)$$

where Φ'_{λ} is the residual neural network that is being trained as a proxy for Δ . For example, if $\Phi'_{\lambda}(X_k, Y_k, N_k) = 0$ then the generator $\Phi_{\lambda}(X_k, Y_k, N_k)$ is indistinguishable from the null hypothesis H_0 , which should be benign. Conversely, the NCD attacks are represented by $\Phi'_{\lambda}(X_k, Y_k, N_k) \neq 0$, corresponding to the deviation to the system $Y_k = \Gamma_{\theta_0}(X_k)$ introduced by the malware.

The network architecture of $\Phi'_{\lambda}(X_k, Y_k, N_k)$ is 5-layer neural network as seen in Table 4.2. The model consists of dense layers connected with dropout layers to improve training on each individual node for learning the signal distribution. The generator and discriminator networks were implemented using Tensorflow with a `tf.Function` block for gradient mapping. We used a set batch size to feed in samples to the generator, then compared those samples with the system samples in the discriminator. To prevent overfitting, a small amount of Gaussian noise is added to the system output before being passed to the generator.

Table 4.1: Sequential Model implemented in Tensorflow for LMP-GAN Discriminator with sigmoid output.

Layer (type)	Output Shape	Param No.
dense	(?, 1, 50)	2550
dense	(?, 1, 500)	25500
dropout	(?, 1, 500)	0
dense	(?, 1, 333)	166833
dropout	(?, 1, 333)	0
dense	(?, 1, 250)	83500
dropout	(?, 1, 250)	0
dense	(?, 1, 50)	12550
dense	(?, 1, 1)	51
Total params: 290,984		
Trainable params: 290,984		
Non-trainable params: 0		

Table 4.2: Sequential Model implemented in Tensorflow for LMP-GAN Generator with output matching system sample size.

Layer (type)	Output Shape	Param No.
dense	(?, 333)	16983
dropout	(?, 333)	0
dense	(?, 500)	167000
dropout	(?, 500)	0
dense	(?, 250)	125250
dense	(?, 50)	12550
dense	(?, 1, 50)	2550
Total params: 324,333		
Trainable params: 324,333		
Non-trainable params: 0		

CHAPTER V

EXPERIMENTAL EVALUATION

There exists no real-world NCD attack datasets since NCD Trojan attacks for signal systems have not yet been reported in the wild (to the best of the authors’ knowledge). For this reason, we assess the effectiveness of the proposed LMP-GAN-based OOD detector in two ways.

- In Section VI, we perform OOD detection on synthetic data, where the training and test dataset are drawn from known statistical distributions. The LMP-GAN is benchmarked against Neyman-Pearson detector in (3.2), which represents the theoretical limit to a binary classifier (learned or analytical alike) that cannot be beaten; as well as state-of-the-art supervised binary classifier.
- In Sections 7.1 and 7.2, we evaluate and benchmark the performance of LMP-GAN as a general-purpose anomaly detector using well known real-world datasets. The purpose of these experiments is train and test the proposed LMP-GAN on datasets representing highly complex and diverse system behavior with no obvious analytical expressions. We benchmark against the state-of-the-art anomaly detection methods.

With the verifiable OOD performance (Section VI) and the demonstrated ability to handle data of complexity and diversity comparable to the cyberphysical systems that LMP-GAN is expected to protect (Sections 7.1 and 7.2), LMP-GAN has a realistic potential to protect against NCD attack in the real-world.

CHAPTER VI

SYNTHETIC OOD DETECTION EXPERIMENTS

Let $I \in \mathbb{R}^{N \times N}$ be an identity matrix. Consider the null and alternative hypotheses of N -dimensional, independent, identically distributed zero-mean, unit variance Laplacian and Gaussian vectors, respectively:

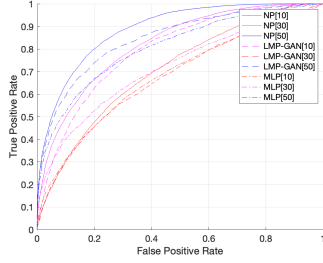
$$\begin{cases} H_0 : & Y \sim \text{Laplace}(0, I) \\ H_1 : & Y \sim \text{Normal}(0, I). \end{cases} \quad (6.1)$$

Figure 6.1(a) shows receiver operator characteristics (ROC) curves corresponding to the proposed LMP-GAN detector (NP: dashed lines), the Neyman-Pearson detector (solid lines), and multilayer perceptron (MLP, dash-dot lines) [7]. The Neyman-Pearson in (3.2) is a provably optimal binary classifier, meaning that it represents the theoretical limit to the detection task in (6.1). MLP is a state-of-the-art supervised binary classification method that performed best in the malware dataset study in Section 7.1.

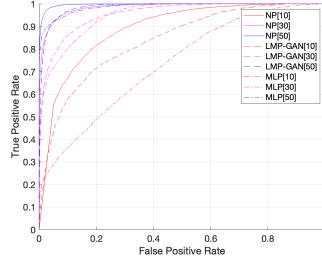
The ability of LMP-GAN to discriminate H_0 and H_1 is less accurate when compared to Neyman-Pearson (unsurprisingly) for the vector lengths of $N=10, 30, 50$. Yet, LMP-GAN outperforms MLP except when the false positive rate is low (though LMP-GAN is very close). These trends are confirmed by the corresponding numerical evaluation in Table 6.1, also showing the strength of LMP-GAN in terms of the areas under the ROC curve, precision, recall, and F-1 scores. As it is not easy for semi-supervised model such as ours trained on H_0 data only to be competitive with a fully supervised model trained on H_0 and H_1 data, it is a remarkable achievement.

We repeat the experiment to assess the discriminators' ability to learn correlation structures within data. Consider the hypothesis test:

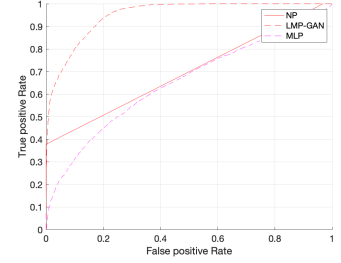
$$\begin{cases} H_0 : & \text{Normal}(0, I) \\ H_1 : & \text{Normal}(0, \Sigma), \end{cases} \quad (6.2)$$



(a) Hypothesis test in (6.1)



(b) Hypothesis test in (6.2)



(c) Hypothesis test in (6.4)

Figure 6.1: Out-of-distribution detection results comparing Neyman-Pearson (NP solid lines, theoretical bound performance) [6], multilayer perceptron (MLP, dash-dot lines, supervised) [7], and the proposed LMP-GAN (dashed lines, semi-supervised). (a) The inliers and outliers are N dimensional Gaussian and Laplacian vectors, respectively. (b) The inlier and outliers are independent and correlated Gaussian vectors, respectively. (c) Result of applying network trained to discriminate H_0 and H_1 in (6.4), tested on H_0 v.s. H_2 . Despite being trained only on inlier samples, it is competitive with the theoretically optimal binary classification result and outperforms MLP trained with inlier and outlier samples.

Table 6.1: The out-of-distribution (OOD) detection results tested on synthetic data. Hypotheses tests in (6.1) and (6.2) are carried out on vectors of lengths $N=10/N=30/N=50$; hypothesis test in (6.4) is conducted for $N=50$. Neyman-Pearson represents the theoretical limit to each binary classification task.

Hypothesis	Method	Accuracy	ROC-AUC	Precision	Recall	F-1
(6.1)	Neyman-Pearson [6]	N/A	0.702/0.819/0.885	0.833/0.968/0.997	0.768/0.931/0.980	0.799/0.949/0.988
	MLP [7]	0.534/0.652/0.702	0.698/0.716/0.807	0.604/0.697/0.715	0.534/0.652/0.702	0.440/0.631/0.697
	LMP-GAN	0.585/0.678/0.793	0.617/0.803/0.863	0.625/0.680/0.767	0.585/0.678/0.762	0.549/0.677/0.685
(6.2)	Neyman-Pearson [6]	N/A	0.869/0.984/0.996	0.826/0.988/0.999	0.259/0.304/0.335	0.394/0.465/0.502
	MLP [7]	0.5105/0.596/0.829	0.733/0.967/0.987	0.752/0.776/0.872	0.510/0.596/0.829	0.356/0.517/0.824
	LMP-GAN	0.683/0.831/0.936	0.836/0.955/0.985	0.758/0.861/0.937	0.683/0.831/0.936	0.659/0.828/0.937
(6.4) (H_0 vs H_2)	Neyman-Pearson [6]	N/A	0.668	0.734	0.602	0.662
	MLP [7]	0.504	0.562	0.731	0.504	0.343
	LMP-GAN	0.618	0.756	0.783	0.618	0.553

where the off-diagonal entry in the covariance matrix $\Sigma \in \mathbb{R}^{N \times N}$ determines the degree of correlation between the vector elements:

$$\Sigma = \begin{bmatrix} 1 & 0.5 & & \\ 0.5 & 1 & 0.5 & \\ & 0.5 & 1 & \ddots \\ & & \ddots & \ddots \end{bmatrix}. \quad (6.3)$$

In Figure 6.1(b), the theoretical limits to discriminate the correlated and uncorrelated vectors is again represented by the Neyman-Pearson. We contrast this to the proposed LMP-GAN, where the discriminator trained on uncorrelated normal vector detects the correlated vector as as OOD/outlier (without being explicitly trained to do so). The performance gap between theoretical limit and the proposed semi-supervised model narrows significantly as the number of samples increase, as is supported also by the quantitative assessment in Table 6.1. LMP-GAN outperformed MLP when the vector length is short ($N=10$); for longer vector lengths ($N=50$), MLP performed better for the low false positive rate, but LMP-GAN was better overall.

In our last synthetic data experiment, we highlight the advantages of OOD detection against novel threats. Consider the following three hypotheses:

$$\begin{cases} H_0 : & \text{Laplace}(0, I) \\ H_1 : & \text{Normal}(0, I) \\ H_2 : & \text{Normal}(0, \Sigma), \end{cases} \quad (6.4)$$

where Σ is the covariance matrix in (6.3). We interpret H_0 and H_1 as the normal and compromised system behavior that supervised learning methods are trained with, respectively, and H_2 as the novel (unseen) malware. Figure 6.1(a) shows the result of discriminating H_0 v.s. H_1 —this scenario is identical to the configuration in (6.1).

How well does a binary classifier designed/trained to discriminate H_0 and H_1 detect a novel malware H_2 not included in the training set? Unsurprisingly, Figure 6.1(c) shows

Neyman-Pearson and MLP fail to achieve the level of performance seen in Figure 6.1(a), due to the mismatch between the designed/trained alternative (H_1) and the unseen alternative (H_2). This example clearly points to the inability of the supervised binary classifier models such as the MLP to protect the signal/data processing systems from novel threats that it was not trained for. By contrast, the proposed LMP-GAN trained with H_0 is able to maintain consistent performance to detect OOD samples in both H_1 and H_2 hypothesis.

CHAPTER VII

APPLIED DATASETS AND RESULTS

7.1 Malware Dataset

The Malware API Dataset in [8] is a popular real-world malware benchmark of high dimensional data representing a highly complex system behavior. The set includes the first 100 non-repeated consecutive API calls from the parent process made by code labeled as *goodware* (normal or H_0) and *malware* (compromised or H_1). There are 42,797 malware and 1,079 goodware files. LMP-GAN was trained on randomly selected 539 training goodware files; and tested on the remaining 540 goodware files and 540 randomly selected malware files. The experiment was repeated 5 times and we report the averaged performance metrics in Table 7.1.

The results were compared to the recent classifier models that have reported performance on the same Malware API dataset. Methods in [7, 56–58] are fully supervised learning binary classifiers that require samples from H_0 and H_1 to perform contrastive learning. Known as *fewshot learning*, methods in [59] perform supervised learning from a limited number of malware samples. Though supervised in the strictest sense, it accommodates the practical challenges of observing large labeled malware data. Besides the proposed LMP-GAN, method in [47] is the only semi-supervised model where the likelihood ratio test is constructed on noise-corrupted H_0 data as a proxy H_1 .

Table 7.1 shows the benchmarking scores. Among the supervised models trained on good/malware, MLP in [7] has the highest accuracy, AUC; DGCNN in [56] has the highest recall F-1 scores, and ExtraTrees in [58] has the highest precision. Despite being a semi-supervised model trained only on goodware, the proposed LMP-GAN is highly competitive,

Table 7.1: The detection performance of LMP-GAN and the state-of-the-art alternatives tested on Malware API Dataset in [8].

Model	Labeled Goodwares	Labeled Malwares	Unlabeled	Accuracy	ROC-AUC	Precision	Recall	F-1
DGCNN [56]	X	X		<u>0.924</u>	<u>0.973</u>	0.913	0.919	0.920
MLP [7]	X	X		0.990	0.987	0.930	0.760	0.820
LSTM [57]	X	X		0.911	0.927	0.842	0.853	0.842
Random Forest [58]	X	X			0.809	<u>0.965</u>	0.809	0.869
CatBoost [58]	X	X			0.820	0.962	0.820	0.877
XGBoost [58]	X	X			0.782	0.956	0.7816	0.8473
ExtraTrees [58]	X	X			0.741	0.970	0.7411	0.8162
NODE [58]	X	X			0.724	0.953	0.7238	0.7971
TabNet [58]	X	X			0.767	0.899	0.7674	0.8166
CNN-LSTM [57]	X	X		0.900	0.960	0.900	0.740	0.810
FewM-HGCL [59]	X	few		0.857		0.876	0.876	0.864
Likelihood ratio [47]	X			0.512	0.723	0.753	0.512	0.359
LMP-GAN (proposed)	X			0.895	0.952	0.896	<u>0.901</u>	<u>0.895</u>

beating majority of fully supervised models in AUC (4rd place), recall (2nd place), and F-1 scores (2nd place). It also outperforms the fewshot method in [59] in every metric.

We may conclude that a state-of-the-art semi-supervised method can be competitive with fully supervised models, even for real-world data representing complex attacks made on cyber-physical systems. As already shown in Section VI, the semi-supervised LMP-GAN is expected to handle novel malware better than the supervised model.

7.2 Credit Card Fraud Dataset

The Credit Card Fraud Dataset in [9] is another benchmarking dataset for evaluating anomaly detection performance. This dataset is comprised of 30-dimensional feature vectors extracted from series of financial transactions—time, amount, and 28-dimensional PCA components extracted from consumer transactional data (to keep the data anonymous). The dataset is imbalanced in the sense that out of 284,807 transactions, there are only 492 fraudulent cases. This is favorable for LMP-GAN as the number H_0 samples is high. For training, randomly selected 50% of the non-fraudulent data was used. Testing was conducted

Table 7.2: The detection performance of LMP-GAN and the state-of-the-art alternatives tested on Credit Card Fraud Dataset in [9].

Model	Labeled Transactions	Labeled Fraud	Unlabeled	Accuracy	ROC-AUC	Precision	Recall	F-1
Decision Tree [60]	X	X		0.997	<u>0.997</u>	0.996	0.998	0.997
Random Forest [61]	X	X		1.0	0.943	<u>0.970</u>	0.890	<u>0.930</u>
AdaBoost [61]	X	X		<u>0.999</u>	0.969	0.891	0.821	0.853
Ensemble Voting [60]	X	X	X			0.890	0.920	0.900
LMP-GAN (proposed)	X			0.997	0.999	0.996	<u>0.996</u>	0.997

on 492 fraudulent cases as well as the 2460 randomly selected samples from the remaining non-fraudulent data.

As evidenced by Table 7.2, LMP-GAN was able to identify the anomaly nearly flawlessly. It performs nearly identically to Decision Tree [60] and outperforms other supervised methods. We may conclude that a large number of H_0 samples contribute to improved anomaly detection quality.

CHAPTER VIII

GAN ABLATION STUDY

We conducted ablation experiments to further understand the specific design choices that make LMP-GAN unique and effective. First, we compared the LMP-GAN structure illustrated in Figure 4.1(b) with the conventional GAN structure shown in Figure 4.1(a). As we report in Table 8.1, the result of training both GAN structures using the same generator and discriminator loss functions in (4.1) and (4.2) yielded comparable results in terms of the area under the curve (AUC), but LMP-GAN had higher F-1 score. This is thanks to the fact that the outlier is modeled as a perturbation to the H_0 system output, making it easier for the generator to achieve its objectives.

Second, we examine the generator loss function (4.1), where we specifically set threshold τ to a non-zero value to find the boundary set $\frac{\partial}{\partial \tau} \Omega_\tau$ containing the adversarial NCD attack point. We contrast this to the conventional *domain expansion* generator loss designed to evade detection, which is akin to minimizing the generator loss in (4.1) with τ set to 0. Indeed, as evidenced by Table 8.1, training GAN structures in Figures 4.1(a) and 4.1(b)

Table 8.1: Ablation study examining specific design choices that make LMP-GAN unique and effective. We compared the GAN network structures (Figure 4.1(a) vs Figure 4.1(b)), generator loss functions (minimax $\tau = 0.5$ vs domain expansion $\tau = 0$), and outlier generations (neural network generator vs noise). All methods trained on length $N=50$ vectors for 2350 epochs.

Hypothesis	Ablation Description	Generator Type	Generator Loss	GAN Structure	ROC-AUC	F-1
(6.1)	Proposed LMP-GAN	Neural Net Φ'_λ	(4.1) $\tau = 0.5$	Figure 4.1(b)	0.863	0.685
	LMP-GAN with conventional generator	Neural Net Φ_λ	(4.1) $\tau = 0.5$	Figure 4.1(a)	<u>0.838</u>	0.379
	Conventional domain expansion	Neural Net Φ_λ	(4.1) $\tau = 0$	Figure 4.1(a)	0.795	0.361
	Domain expansion with Figure 4.1(b)	Neural Net Φ'_λ	(4.1) $\tau = 0$	Figure 4.1(b)	0.782	0.471
	Replace generator with additive noise [47]	Noise	N/A	Figure 4.1(b)	0.819	<u>0.695</u>
(6.2)	Proposed LMP-GAN	Neural Net Φ'_λ	(4.1) $\tau = 0.5$	Figure 4.1(b)	0.985	0.937
	Replace generator with additive noise [47]	Noise	N/A	Figure 4.1(b)	<u>0.541</u>	0.333

with τ set to zero worsened the OOD performance significantly. This ablation result is consistent with the observations made in [3, 4, 54] that *domain expansion*-based OOD detection increases high risk of type 1 errors.

Finally, LMP-GAN shares some similarities with the OOD method in [47]—the latter adds noise to perturb H_0 samples instead of training the generator Φ'_λ specifically for this task. To wit, we replaced the generator in Figure 4.1(b) with an additive white Gaussian noise (AWGN) generator, yielding a “non-adversarial” alternative (i.e. noise generator is agnostic to the discriminator performance) to the LMP-GAN. However, the result of training the discriminator using the same loss function (4.2) yielded mixed result, as reported in Table 8.1. Hypothesis testing in (6.1) using AWGN generator worked in its favor since the alternative hypothesis H_1 was also normally distributed. But the performance was unacceptable for hypothesis testing in (6.2) because AWGN is uncorrelated, unlike the H_1 . Thus we conclude that the generator Φ'_λ in Figure 4.1(b) is indeed learning meaningful perturbations to H_0 samples to yield outliers that resemble the minimax/locally most powerful (LMP) test.

CHAPTER IX

MODEL ARCHITECTURE AND TUNING

The proposed model tested in section VI showed how a GAN using only two dense fully connected models could achieve near theoretical limits in binary detection. This was done using only one class, while the generator produced fake synthetic samples to differentiate from the real true samples. Given how simple dense models are to set up, little planning had gone into the architecture. The problem was solved by supplying enough layers and modifying the learning rates for each model. This left much to be desired during training times and model convergence.

The GAN architecture is very fragile; given that the model is adversarial, it's competitive, and each piece of the system can't outpace the other. Controlling the learning rate is one of the best ways to control this pace. Another method is the actual layers themselves. While the systems used for this proposed LMP-GAN used dense layers, given that the data is being treated as a tabular time-based signal, it can be assumed that convolution would be a faster method for learning and potentially classifying the model. Setting up each piece of the model, the generator, and the discriminator with a convolution architecture would benefit the tasks for which they are each responsible.

The generator would be the first of the models to be modified and tasked with generating fake images from the seeded real samples. The proposed LMP-GAN uses dense layers with various dropout layers to prevent overfitting for this task. The task could be done using a U-Net architecture with a 1D Convolution Encoder/Decoder Pair [62]. The U-Net architecture uses the contraction or extraction phase and an expansion phase. In the extraction phase, the encoder, using various 1D-Conv layers and Pooling layers, extracts the features from the signal; at the lowest point, the decoder then attempts to reconstruct the sample in

the expansion phase, with various 1D-Conv layers and Up-sampling layers. These features are then flattened and used with dense layers of thickness to varying degrees in different sources. For this research, the goal is to catch as many of the changes between the different samples in the Convolution layers as possible. During the generation of that sample, the most important ones drive the creation of the new samples. The level of needed dense layers would determine to what degree that can be completely captured by the Convolution layers.

After convolution, the generator will be split into two different models to determine the need for dense layers. The first model, seen in 9.1 shows the U-Net architecture for a Generator with an input vector $[N]$ of 50. Each layer filter stepped up from 50, 100, to 200, then back down, recreating the initial samples. Those layer outputs are flattened before moving on to the dense layers. After the flattening, the first of the two models uses four dense and drop-out layers. This is where the difference for the second "lean" model comes into play, as seen in 9.2, where there is a single dense output after the flattening layer. This puts all the work on the U-Net during the training but could show greater training time, allowing for more complex models in non-toy applications. This also reduced the total parameters in the model, but this can alter the ability of the GAN to converge to a stable adversarial point.

The next change to the models is to look at how the discriminator architecture can be improved. Given that the discriminator is the most important of the two models in the LMP-GAN structure, we want a robust model that can classify outliers well. The proposed change to the discriminator is the application of an Encoder to extract features before the dense layers. Similar to the U-Net architecture, but without the decoder, the encoder uses various 1D Convolution and Pooling layers, but there is no reconstruction. Instead, the features are flattened and passed on to various dense layers.

Table 9.1: Sequential Model implemented in TensorFlow for LMP-GAN Generator with sigmoid output using a U-Net architecture with 1-Dimensional Convolution.

Layer (type)	Output Shape	Param No.
GaussianNoise	(?, 1, 50)	0
conv1d	(?, 46, 50)	300
conv1d	(?, 44, 50)	7550
MaxPooling1d	(?, 22, 50)	0
dropout	(?, 22, 50)	0
conv1d	(?, 18, 100)	25100
conv1d	(?, 16, 100)	30100
MaxPooling1d	(?, 8, 100)	0
dropout	(?, 8, 100)	0
conv1d	(?, 4, 200)	100200
conv1d	(?, 2, 200)	120200
MaxPooling1d	(?, 1, 200)	0
dropout	(?, 1, 200)	0
conv1d	(?, 1, 200)	40200
conv1d	(?, 1, 200)	40200
UpSampling1d	(?, 4, 200)	0
conv1d	(?, 4, 100)	20100
conv1d	(?, 4, 100)	10100
UpSampling1d	(?, 16, 100)	0
conv1d	(?, 16, 50)	5050
conv1d	(?, 16, 50)	2550
UpSampling1d	(?, 48, 50)	0
flatten	(?, 2400)	0
dense	(?, 400)	960400
dropout	(?, 400)	0
dense	(?, 300)	120300
dropout	(?, 300)	0
dense	(?, 200)	60200
dense	(?, 50)	10050
Total params: 1,552,600		
Trainable params: 1,552,600		
Non-trainable params: 0		

Table 9.2: Sequential Model implemented in TensorFlow for LMP-GAN Generator with sigmoid output using a "Lean" UNet architecture with 1-Dimensional Convolution. Lean is defined as removing the Dense post layers from the Conv1D U-net Architecture.

Layer (type)	Output Shape	Param No.
GaussianNoise	(?, 1, 50)	0
conv1d	(?, 46, 50)	300
conv1d	(?, 44, 50)	7550
MaxPooling1d	(?, 22, 50)	0
dropout	(?, 22, 50)	0
conv1d	(?, 18, 100)	25100
conv1d	(?, 16, 100)	30100
MaxPooling1d	(?, 8, 100)	0
dropout	(?, 8, 100)	0
conv1d	(?, 4, 200)	100200
conv1d	(?, 2, 200)	120200
MaxPooling1d	(?, 1, 200)	0
dropout	(?, 1, 200)	0
conv1d	(?, 1, 200)	40200
conv1d	(?, 1, 200)	40200
UpSampling1d	(?, 4, 200)	0
conv1d	(?, 4, 100)	20100
conv1d	(?, 4, 100)	10100
UpSampling1d	(?, 16, 100)	0
conv1d	(?, 16, 50)	5050
conv1d	(?, 16, 50)	2550
UpSampling1d	(?, 48, 50)	0
flatten	(?, 2400)	0
dropout	(?, 400)	0
dense	(?, 50)	120050
Total params: 521,700		
Trainable params: 521,700		
Non-trainable params: 0		

Table 9.3: Sequential Model implemented in Tensorflow for LMP-GAN Discriminator with sigmoid output using an Encoder architecture with 1-Dimensional Convolution.

Layer (type)	Output Shape	Param No.
reshape(reshape)	(?, 50, 1)	0
conv1d	(?, 48, 100)	400
conv1d	(?, 46, 100)	30100
MaxPooling1d	(?, 23, 100)	0
dropout	(?, 23, 100)	0
conv1d	(?, 21, 200)	60200
conv1d	(?, 19, 200)	120200
MaxPooling1d	(?, 9, 200)	0
dropout	(?, 9, 200)	0
conv1d	(?, 7, 400)	240400
conv1d	(?, 5, 400)	480400
MaxPooling1d	(?, 2, 400)	0
flatten	(?, 800)	0
dense	(?, 300)	160200
dropout	(?, 300)	0
dense	(?, 200)	30150
dropout	(?, 200)	0
dense	(?, 100)	15100
dropout	(?, 200)	0
dense	(?, 50)	5050
dense	(None, 1)	51
Total params: 1,142,251		
Trainable params:1,142,251		
Non-trainable params: 0		

The changes to the architecture for the discriminator can be seen in 9.3, showing a three-layer encoder and five dense layers. Given that the discriminator is the more important model of the two for the final product, there isn't a leaner version to test, as the additional dense layers are needed with the encoder to ensure the output quality for anomaly detection.

Various versions of the models are tested in different configurations; given that the GAN model is adversarial, it isn't guaranteed that the models will work well paired with one another or that something is lost that is critical for training. So, each change will be tested against the toy test in hypothesis 6.2 using the Gaussian and correlated Gaussian signals.

Table 9.4: Architecture changes to the proposed GAN by modification of the generator and discriminator using Convolutional Neural networks. Each method tests a structural change using 6.2 to test the effects of CNNs on LMP-GAN performance. False Positive (FP) rates at .01 and .001 are considered for each model as a testing parameter, given that the model’s effectiveness is based on novel malware detection.

Architecture	Generator	Discriminator	ROC-AUC	F-1	Epochs	FP [.01]	FP [.001]
Dense LMP-GAN	Dense NN 4.2	Dense NN 4.1	0.985	0.937	2350	0.711	0.227
Fully Convolution LMP-GAN	U-Net 9.1	Encoder 9.3	0.947	0.860	350	0.476	0.207
Generator 1-D Conv, Stride 3	U-Net 9.1	Dense NN 4.1	0.963	0.832	850	0.389	0.092
Generator 1-D Conv, Stride 5	U-NET 9.1	Dense NN 4.1	0.646	0.336	400	0	0
Discriminator 1-D Conv, Stride 3	Dense NN 4.2	Encoder 9.3	0.954	0.893	1400	0.290	0.046
Gen and Disc 1-D Conv, Stride 3	Lean U-Net 9.2	Encoder 9.3	0.829	0.746	1600	0.160	0.039

This test works well with the 1D Convolution layers that have been added and can be shown within the limits of the Neyman-Pearson. The first experiment will test the U-Net architecture with various Strides using the proposed Dense Discriminator. Then, the new encoder architecture will be tested using the proposed Dense Generator before testing the Full Convolution combination model and then the lean version. All results are recorded in table 9.4 and show the ROC-AUC, the F-1 score for that test, and the number of epochs before further training stopped improving the model. The Epochs show the model peak for each change in the architecture and a relative time increase/decrease for each change. All ROC-AUCs are also plotted in 9.1 to compare each change clearly.

The results defined in figure 9.1 and table 9.4 show no clear improvement to detection capabilities by adding Convolution to either of the parts that make up the LMP-GAN. The only clear improvement that can be seen is in training time, which shouldn’t be overlooked. One of the most difficult parts of the GAN process is the training and production of a model that will converge. It is tedious, and if a model doesn’t converge quickly, it can be a lot of wasted time. Additionally, while this is a simple toy model when using real data, the number of variables can be vastly larger to have a picture of normal system behavior. This alone

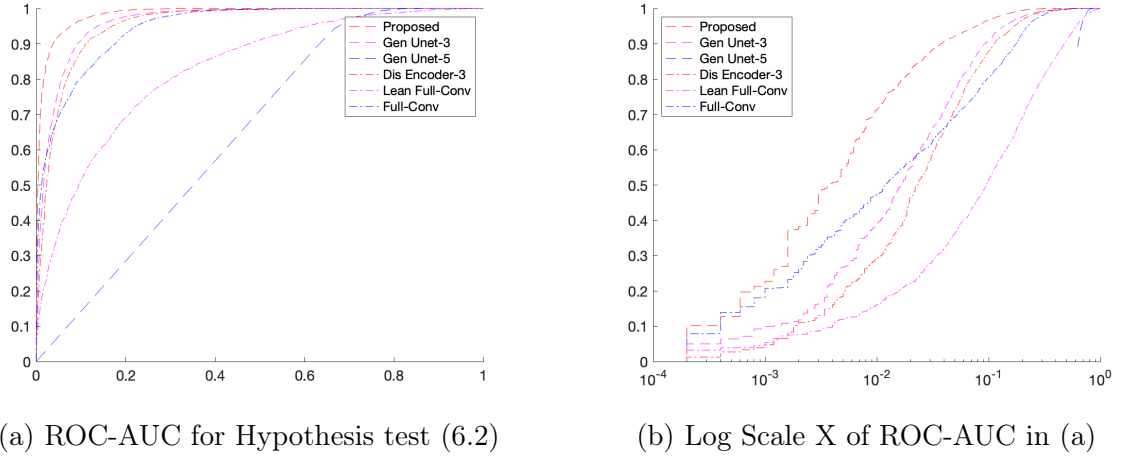


Figure 9.1: Plot of 9.4 results of each architectural change done to the LMP-GAN and the impact on ROC and detection/False-Positive rate. (a) shows the standard ROC-AUC where some models clearly underperform for the task, like "Gen Unet-5" and "Lean Full Conv." The architecture changes are much closer in the 0 to 0.1 False positive range. (b) By applying the log scale to the X-axis, the performance of the various models, at .01 and .001, becomes easier to notice. "Full-Conv" being the second best when compared to the Proposed model.

will account for longer training times, so providing that convolution may be able to improve that should not be discounted. The second best detection results are that of the U-NET generator with an ROC-AUC of 0.963 but a much lower F-1 score of 0.832. Next behind that is the Encoder Discriminator with a dense generator with an ROC-AUC of 0.954 and F-1 of .893. Between the two results, the performance of the second is more notable, with an F-1 score closer to that of the proposed model at 0.937. When looking at the Epochs, the most significant improvement is that of the Full convolution model, using both the U-Net and Encoder architectures, at only 350 epochs compared to the proposed model at 2350. That is 2000 epochs faster, nearly 1/10th the speed of the original model in training time, but with a small reduction in performance.

The results show a possibility of applying a model that is easier to train than the proposed model at the cost of some performance. Given the scale of the task, the LMP-GAN may best be used and deployed in a convolution set-up to achieve convergence. It's not guaranteed that every problem set can clearly be separated between known behavior and anomalies to that behavior. The more complex that space becomes, the more difficult the model is to train, but the results show that modifications can be made to overcome that training barrier.

CHAPTER X

FUTURE AND RELATED WORK

10.1 Future Work

Section 10.1 lays out the next steps for the research and the possible improvements that can be made to the LMP-GAN. While many variations of GANs and anomaly detections exist, few methods exist that use a single class input, so it's important to consider all the possible edge cases around this work and where the LMP-GAN may have difficulties.

10.1.1 Generative Models

The LMP-GAN is based on designing a binary classifier that can adequately detect samples outside of a given variance, meaning that this work is transitional to other generative models. For this work, the GAN has been used for its straightforward application and robustness from its adversarial layout, but other models like the diffusion and even Auto-encoders could theoretically be used. Future work would be building these different generative methods to create pseudo samples around real samples to train out anomalies in a more powerful binary classifier.

One model type gaining popularity is diffusion probabilistic models [63]. Specifically because of their ability to generate new content in various different fields [64] [65] [66]. Variations of this model have produced many of the most popular algorithms that are being seen for deep-fake images or derived artistic prompts trained on publicly accessible works. The diffusion model takes a different approach than a GAN, as the basis for its generative properties is based on breaking down a sample and then probabilistic trying to piece it back together [63]. This allows it to estimate the posterior and priors. The models have shown

that they can handle very detailed data and patterns, and using a method like this may cover areas where the GAN may not be able to generate new samples adequately.

Another type of model that can be used for data generation is an autoencoder. These have a wide range of uses, including anomaly detection, which will be expanded in section 10.1.2. For the purpose of data generation, an auto-encoder is trained to copy input into output. This is done by deconstructing the input data with an encoder's most basic features and then attempting to reconstruct it with a decoder. A trained autoencoder may not be able to create exact copies of its input, but similar to the GAN, it can make fake samples close enough to the input that a binary classifier should be able to use the generated samples to train.

To round out this research and approach fully, different variations of the generation will be tested to see how they change the development of the binary classifier. The novelty of the approach is that it only uses a single class, so the expansion of that class and the ability to morph and change it ultimately gives the classifier the ability to detect anomalies. Documenting how close each generation method gets to creating fake samples and what they may achieve over others could help thwart adversarial attacks, such as non-control data attacks.

10.1.2 Anomaly Detection

Mixed of Experts and Ensemble Models Given that so many possible generative models exist, and each of them may possibly expand on the domain in a way that the others can't, a suggestion for future work to improve the capabilities is to employ an Ensemble or Mixture of Experts (MOE) approach. This would greatly expand the size of the models used, but training time but if training time isn't a factor. This could improve the accuracy of the

models deployed and reduce the possible blind spots they have. The advantages of an Ensemble and a MOE differ depending on the application they are used in. An ensemble would be a larger model that would use several models that would all predict a large task or complex task like anomaly detection; this would also allow other powerful methods that have been benchmarked in the paper to be used with the LMP-GAN. Models trained to target some known families of malware attacks could be paired with the LMP-GAN, reducing the number of Detection samples it would have to worry about if the only worry were detecting unknown samples. The other approach would be to use the MOE, which would train several models on subtasks comprising larger tasks. Such as family types, and only apply the models to the specific task in which they excel. If further research discussed in section 10.1.1 was done and it was found that certain generative models were better at picking out very specific types of attacks, this type of model would be employed for the sub-task. This would reduce the overall size of each needed model, with the goal of deploying the best model for the application and task based on what is known about the application domain space.

Circling back to the original problem that this novel approach is trying to solve, it is the detection of unknown attacks. Cyber attacks are ever-evolving and changing and employ many different attacks, so many that many different subfamilies make up these attacks. The proposed model is an anomaly detector that works as a binary detector, separating normal behavior from malicious anomalies or abnormalities in a signal system. After the first task of separation is done, a new task is needed to separate the anomalies, identify any novel attacks, and separate them from older attacks, which a signature-based method would be able to identify. Future work and development would be best placed around expanding on this capability when considering the application's use of the LMP-GAN, as this will also

aid in user understanding of type I and II errors that will be persistent within the detection task while attempting to minimize as many false positives as possible.

10.2 Related Work

The idea for this work comes from the field of virology, a subsection of microbiology and pathology. In Virology, identifying the types of viruses that can harm biological systems often considers the types of viruses that have existed and the immune systems' ability to replicate that virus. Scientists will collect viruses to make vaccines, but to create one, they must make new versions and predict how that virus will mutate. Biological systems, like synthetic systems, behave in homeostatic regions. They will react to foreign bodies that enter the body as a defense when that homeostasis is interrupted, or in other words, that system behavior becomes abnormal. We have evolved as a species to develop vaccines as tools to fight viruses, but the body also has a defense mechanism and antibodies to deal with foreign agents. The body has to be skilled at understanding what is part of itself or part of the normal system and what doesn't belong, as well as how to respond to it when it enters that space. This inspiration led to the LMP-GAN, which also considers normal system behavior and variations of known behaviors to determine better what belongs and doesn't.

Relatively close to the approach in this paper is the study done on few-shot samples. In cyber-security, the few-shot approach attempts to make a classifier that can expand and detect using a very small number of samples for a class type. This is not technically training a model for an "Unknown" attacker but training it for an attacker you know very little about. Much of this work could potentially be used to aid the LMP-GAN. In addition, if, in the application, the LMP-GAN detects novel attacks, a few-shot model may be the best

type of classifier to use immediately after to detect that type of attack until more samples are found. Given this work’s closeness and the problem’s difficulty and nature, a few-shot model was included in the malware study 7.1.

While most of this paper has talked about the implications this work has for the field of cyber security and NCD attacks specifically, the truth is this novel approach to anomaly detection has many applications outside of that. The key is that this new method specializes in detecting a single class. Data is often imbalanced, and when events are incredibly rare, it can’t be hard to detect them or have enough examples for positive detections. Additionally, when technology is so new, it’s not always clear what to look for. This makes LMP-GAN a perfect application for these scenarios. One example would be Fault testing in control systems, where parts or systems have to be made and performed in a very specific manner; this normal behavior could possibly be developed and tested, and when a part goes outside of those ranges, it would be flagged as a default. In massive industries like avionics, where aircraft parts and other systems cost so much money and they may sit around for a while, this application could even tell if the sensor in the system is not operating properly, aiding in the detection of operational faults. Additionally, complex systems like voltage or temperature monitoring of a large system state is another place where LMP-GAN could be applied, as you know the regular temperatures and spikes that exist within the process that takes place, but want to be aware of catastrophic anomalies that could destroy or damage systems.

Another promising application is medical systems, where patients and individuals must be monitored, or their systems must be set to a specific range before they go off. These are often hard-coded; some controllers look promising that they learn based on biology and can be personalized, but they often require much data to produce. Given the nature of the LMP-GAN and how it augments and generates data, it could be used for different systems

to maintain normal behavior and flag anomalies that appear on an EKG or a respirator. It's even possible that it could be even more sensitive to the unique anomalies that exist for the individual than what a trained professional may be able to detect.

Many more examples of systems would benefit from the LMP-GAN and, more yet, from the variation that may be produced from future work around other generative models. Developing secure and fault-free systems will benefit many fields if the LMP-GAN proves to be diverse enough to be a solution to be applied within them.

CHAPTER XI

CONCLUSION

In this work, we considered a new methodology for protecting cyber-physical signal systems from a novel Trojan threat called a non-control data (NCD) attack. Specifically, we formulated the detection task as a discriminator-side minimax optimization problem to minimize the difference between the normal and the compromised system outputs. The proposed LMP-GAN is an out-of-distribution (OOD) detection method using the generative adversarial network to train on the more inline (system behaving correctly) samples only. Theoretically, it can be interpreted as a machine learning variant of the locally most powerful composite hypothesis test.

Owing to the lack of known NCD attacks and datasets, we used synthetic data to verify LMP-GAN's OOD performance and real-world malware and credit card fraud data to prove LMP-GAN's ability to handle data of complexity and diversity comparable to the cyber-physical systems that it is expected to protect. The synthetic data experiment empirically proved that the LMP-GAN narrows the performance gap with the theoretically optimal binary classifier task as the number of samples increases. It is robust to novel (unseen) attacks because it only trains on the null hypothesis samples. We may conclude that the LMP-GAN model is helpful in signal/data processing applications where considering all possible vulnerabilities to that system is impractical [67, 68]. In real-world data benchmarking, LMP-GAN was competitive with the best-performing supervised methods despite training with inlier samples only while beating many state-of-the-art supervised binary classifiers.

As a future direction in research, the proposed LMP-GAN framework currently lacks a way to train for known threats. In order to be more broadly useful in application space, it would be preferable to train the classifier with samples drawn from null (normal system

behavior, or H_0) and known threats (already observed malware, or H_1) in a fully supervised manner, but yet leverage the LMP framework to protect against novel threats (unseen malware, or H_2) in a semi-supervised manner [69]. By explicitly training for H_1 class, it is expected to improve the detection rate for known threats even while being robust to novel NCD attacks. Also, LMP-GAN may be viewed as a general-purpose semi-supervised minimax problem solver. We plan to investigate other applications where minimax-based out-of-distribution detection is appropriate.

BIBLIOGRAPHY

- [1] D. P. Kingma and P. Dhariwal, “Glow: Generative flow with invertible 1x1 convolutions,” *Advances in neural information processing systems*, vol. 31, 2018.
- [2] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” *arXiv preprint arXiv:1312.6114*, 2013.
- [3] E. Nalisnick, A. Matsukawa, Y. W. Teh, D. Gorur, and B. Lakshminarayanan, “Do deep generative models know what they don’t know?” *arXiv preprint arXiv:1810.09136*, 2018.
- [4] K. Kim, J. Shin, and H. Kim, “Locally most powerful bayesian test for out-of-distribution detection using deep generative models,” in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 14 913–14 924.
- [5] A. Van den Oord, N. Kalchbrenner, L. Espeholt, O. Vinyals, A. Graves *et al.*, “Conditional image generation with pixelcnn decoders,” *Advances in neural information processing systems*, vol. 29, 2016.
- [6] J. Neyman and E. S. Pearson, “Contributions to the theory of testing statistical hypotheses.” *Statistical research memoirs*, 1936.
- [7] NIKOLASCH. (2022) Mlp model: Malware detection. [Online]. Available: <https://www.kaggle.com/code/nikolasch/mlp-model-malware-detection>
- [8] A. OLIVEIRA. (2020) Malware analysis datasets: Api call sequences. [Online]. Available: <https://www.kaggle.com/datasets/ang3loliveira/malware-analysis-datasets-api-call-sequences>
- [9] M. L. GROUP. (2018) Credit card fraud detection. [Online]. Available: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud?resource=download>
- [10] R. Richmond, “Malware hits computerized industrial equipment,” *New York Times*. [Online]. Available: <https://archive.nytimes.com/bits.blogs.nytimes.com/2010/09/24/malware-hits-computerized-industrial-equipment>
- [11] A. A. Elkhail, R. U. D. Refat, R. Habre, A. Hafeez, A. Bacha, and H. Malik, “Vehicle security: A survey of security issues and vulnerabilities, malware attacks and defenses,” *IEEE Access*, vol. 9, pp. 162 401–162 437, 2021.
- [12] S. Chen, J. Xu, E. C. Sezer, P. Gauriar, and R. K. Iyer, “Non-control-data attacks are realistic threats.” in *USENIX security symposium*, vol. 5, 2005, p. 146.

- [13] C. Schlesinger, K. Pattabiraman, N. Swamy, D. Walker, and B. Zorn, “Modular protections against non-control data attacks,” *Journal of Computer Security*, vol. 22, no. 5, pp. 699–742, 2014.
- [14] T. Abera, N. Asokan, L. Davi, J.-E. Ekberg, T. Nyman, A. Paverd, A.-R. Sadeghi, and G. Tsudik, “C-flat: control-flow attestation for embedded systems software,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 743–754.
- [15] H. Hu, S. Shinde, S. Adrian, Z. L. Chua, P. Saxena, and Z. Liang, “Data-oriented programming: On the expressiveness of non-control data attacks,” in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 969–986.
- [16] H. Hu, Z. L. Chua, S. Adrian, P. Saxena, and Z. Liang, “Automatic generation of {Data-Oriented} exploits,” in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 177–192.
- [17] “Rops are for the 99%,” *CanSecWest 2014*. [Online]. Available: https://github.com/tombkeeper/Talks/blob/master/CanSecWest_2014/ROPs_are_for_the_99%25_%5BCSW2014%5D.pdf
- [18] “Openssl ‘heartbleed’ vulnerability (cve-2014-0160),” *United States Cybersecurity and Infrastructure Security Agency*. [Online]. Available: <https://www.cisa.gov/news-events/alerts/2014/04/08/openssl-heartbleed-vulnerability-cve-2014-0160>
- [19] H. Ye, S. Liu, Z. Zhang, and H. Hu, “VIPER: Spotting Syscall-Guard variables for Data-Only attacks,” in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 1397–1414. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/ye>
- [20] C. Zhang, T. Wei, Z. Chen, L. Duan, L. Szekeres, S. McCamant, D. Song, and W. Zou, “Practical control flow integrity and randomization for binary executables,” in *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, pp. 559–573.
- [21] N. Carlini, A. Barresi, M. Payer, D. Wagner, and T. R. Gross, “{Control-Flow} bending: On the effectiveness of {Control-Flow} integrity,” in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 161–176.
- [22] C. Tice, T. Roeder, P. Collingbourne, S. Checkoway, Ú. Erlingsson, L. Lozano, and G. Pike, “Enforcing {Forward-Edge}{Control-Flow} integrity in {GCC} & {LLVM},” in *23rd USENIX security symposium (USENIX security 14)*, 2014, pp. 941–955.
- [23] M. Zhang and R. Sekar, “Control flow and code integrity for cots binaries: An effective defense against real-world rop attacks,” in *Proceedings of the 31st Annual Computer Security Applications Conference*, 2015, pp. 91–100.
- [24] B. Niu and G. Tan, “Per-input control-flow integrity,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 914–926.

- [25] —, “Modular control-flow integrity,” *SIGPLAN Not.*, vol. 49, no. 6, p. 577–587, jun 2014. [Online]. Available: <https://doi.org/10.1145/2666356.2594295>
- [26] J. Criswell, N. Dautenhahn, and V. Adve, “Kcofi: Complete control-flow integrity for commodity operating system kernels,” in *2014 IEEE symposium on security and privacy*. IEEE, 2014, pp. 292–307.
- [27] D. Bigelow, T. Hobson, R. Rudd, W. Streilein, and H. Okhravi, “Timely rerandomization for mitigating memory disclosures,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’15. New York, NY, USA: Association for Computing Machinery, 2015, p. 268–279. [Online]. Available: <https://doi.org/10.1145/2810103.2813691>
- [28] A. J. Mashtizadeh, A. Bittau, D. Boneh, and D. Mazières, “Ccfi: Cryptographically enforced control flow integrity,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’15. New York, NY, USA: Association for Computing Machinery, 2015, p. 941–951. [Online]. Available: <https://doi.org/10.1145/2810103.2813676>
- [29] G. Torres, Z. Yang, Z. Blasingame, J. Bruska, and C. Liu, “Detecting non-control-flow hijacking attacks using contextual execution information,” in *Proceedings of the 8th International Workshop on Hardware and Architectural Support for Security and Privacy*, 2019, pp. 1–8.
- [30] X. Nie, L. Chen, H. Wei, Y. Zhang, N. Cui, and G. Shi, “Kpdfi: Efficient data flow integrity based on key property against data corruption attack,” *Computers & Security*, vol. 128, p. 103183, 2023.
- [31] M. Costa, M. Castro, and T. Harris, “Securing software by enforcing data flow integrity,” Jul. 12 2016, uS Patent 9,390,261.
- [32] M. Geden and K. Rasmussen, “Regguard: Leveraging cpu registers for mitigation of control-and data-oriented attacks,” *Computers & Security*, vol. 129, p. 103213, 2023.
- [33] S. Chen, J. Xu, N. Nakka, Z. Kalbarczyk, and R. Iyer, “Defeating memory corruption attacks via pointer taintedness detection,” in *2005 International Conference on Dependable Systems and Networks (DSN’05)*, 2005, pp. 378–387.
- [34] E. Göktas, A. Economopoulos, R. Gawlik, E. Athanasopoulos, G. Portokalidis, and H. Bos, “Bypassing clang’s safestack for fun and profit,” *Black Hat Europe*, p. 21, 2016.
- [35] K. K. Ispoglou, B. AlBassam, T. Jaeger, and M. Payer, “Block oriented programming: Automating data-only attacks,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1868–1882.

- [36] L. Cheng, S. Ahmed, H. Liljestrand, T. Nyman, H. Cai, T. Jaeger, N. Asokan, and D. Yao, "Exploitation techniques for data-oriented attacks with existing and potential defense approaches," *ACM Transactions on Privacy and Security (TOPS)*, vol. 24, no. 4, pp. 1–36, 2021.
- [37] X. Shu, D. Yao, and B. G. Ryder, "A formal framework for program anomaly detection," in *Research in Attacks, Intrusions, and Defenses: 18th International Symposium, RAID 2015, Kyoto, Japan, November 2-4, 2015. Proceedings 18*. Springer, 2015, pp. 270–292.
- [38] R. Sekar, M. Bendre, D. Dhurjati, and P. Bollineni, "A fast automaton-based method for detecting anomalous program behaviors," in *Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001*. IEEE, 2000, pp. 144–155.
- [39] D. Wagner and R. Dean, "Intrusion detection via static analysis," in *Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001*. IEEE, 2000, pp. 156–168.
- [40] F. Stephanie, "A sense of self for unix processes," in *IEEE Symposium on Security and Privacy, 1996*, 1996, pp. 120–128.
- [41] J.-C. Demay, E. Totel, and F. Tronel, "Sidan: A tool dedicated to software instrumentation for detecting attacks on non-control-data," in *2009 Fourth International Conference on Risks and Security of Internet and Systems (CRiSIS 2009)*, 2009, pp. 51–58.
- [42] A. K. Alnaim, "Misuse patterns from the threat of modification of non-control data in network function virtualization," *Future Internet*, vol. 14, no. 7, p. 201, 2022.
- [43] S. Bhatkar, A. Chaturvedi, and R. Sekar, "Dataflow anomaly detection," in *2006 IEEE Symposium on Security and Privacy (S&P'06)*, 2006, pp. 15 pp.–62.
- [44] S. Lu, M. Seo, and R. Lysecky, "Timing-based anomaly detection in embedded systems," in *The 20th Asia and South Pacific Design Automation Conference*. IEEE, 2015, pp. 809–814.
- [45] R. C. Aygun and A. G. Yavuz, "Network anomaly detection with stochastically improved autoencoder based models," in *2017 IEEE 4th International Conference on Cyber Security and Cloud Computing (CSCloud)*, 2017, pp. 193–198.
- [46] M. S. Akhtar and T. Feng, "Detection of malware by deep learning as cnn-lstm machine learning techniques in real time," *Symmetry*, vol. 14, no. 11, 2022. [Online]. Available: <https://www.mdpi.com/2073-8994/14/11/2308>
- [47] J. Ren, P. J. Liu, E. Fertig, J. Snoek, R. Poplin, M. Depristo, J. Dillon, and B. Lakshminarayanan, "Likelihood ratios for out-of-distribution detection," *Advances in neural information processing systems*, vol. 32, 2019.
- [48] S. L. Isaacson, "On the Theory of Unbiased Tests of Simple Statistical Hypotheses Specifying the Values of Two or More Parameters," *The Annals of*

- Mathematical Statistics*, vol. 22, no. 2, pp. 217 – 234, 1951. [Online]. Available: <https://doi.org/10.1214/aoms/1177729642>
- [49] H. V. Poor, *An introduction to signal detection and estimation*. Springer Science & Business Media, 1998.
 - [50] R. Veldhuis and D. Zeng, “On neyman-pearson optimality of binary neural net classifiers,” 2021.
 - [51] U. Müller-Funk, F. Pukelsheim, and H. Witting, “Locally most powerful tests for two-sided hypotheses,” in *Probability and statistical decision theory: proceedings of the 4th Pannonian Symposium on Mathematical Statistics, Bad Tatzmannsdorf, Austria, 4-10 Sept. 1983, vol. A*, F. Konecny, J. Mogyoródi, and W. Wertz, Eds., 1985.
 - [52] M. Omelka, “The behavior of locally most powerful tests,” *Kybernetika*, vol. 41, no. 6, pp. 699–712, 2005.
 - [53] A. Sen Gupta and L. Vermeire, “Locally optimal tests for multiparameter hypotheses,” 1986.
 - [54] Z. Xiao, Q. Yan, and Y. Amit, “Likelihood regret: An out-of-distribution detection score for variational auto-encoder,” *Advances in neural information processing systems*, vol. 33, pp. 20 685–20 696, 2020.
 - [55] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
 - [56] A. Schranko de Oliveira and R. J. Sassi, “Behavioral malware detection using deep graph convolutional neural networks,” 2019.
 - [57] M. DALY. (2021) Behavioral malware detection with cnn-lstm. [Online]. Available: <https://www.kaggle.com/code/maidaly/behavioral-malware-detection-with-cnn-lstm>
 - [58] A. Cannarile, V. Dentamaro, S. Galantucci, A. Iannaccone, D. Impedovo, and G. Pirlo, “Comparing deep learning and shallow learning techniques for api calls malware prediction: a study,” *Applied Sciences*, vol. 12, no. 3, p. 1645, 2022.
 - [59] C. Liu, B. Li, J. Zhao, Z. Zhen, X. Liu, and Q. Zhang, “Fewm-hgcl: Few-shot malware variants detection via heterogeneous graph contrastive learning,” *IEEE Transactions on Dependable and Secure Computing*, 2022.
 - [60] UVINIR. (2022) Credit card fraud detection | decision trees. [Online]. Available: <https://www.kaggle.com/code/uvinir/credit-card-fraud-detection-decision-trees>
 - [61] R. Sailusha, V. Gnaneswar, R. Ramesh, and G. R. Rao, “Credit card fraud detection using machine learning,” in *2020 4th international conference on intelligent computing and control systems (ICICCS)*. IEEE, 2020, pp. 1264–1270.

- [62] J. S. Suri, M. Bhagawati, S. Agarwal, S. Paul, A. Pandey, S. K. Gupta, L. Saba, K. I. Paraskevas, N. N. Khanna, J. R. Laird *et al.*, “Unet deep learning architecture for segmentation of vascular and non-vascular images: a microscopic look at unet components buffered with pruning, explainable artificial intelligence, and bias,” *Ieee Access*, vol. 11, pp. 595–645, 2022.
- [63] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” in *International conference on machine learning*. PMLR, 2015, pp. 2256–2265.
- [64] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” *arXiv preprint arXiv:2010.02502*, 2020.
- [65] L. Ruan, Y. Ma, H. Yang, H. He, B. Liu, J. Fu, N. J. Yuan, Q. Jin, and B. Guo, “Mm-diffusion: Learning multi-modal diffusion models for joint audio and video generation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 10 219–10 228.
- [66] J. Ho, T. Salimans, A. Gritsenko, W. Chan, M. Norouzi, and D. J. Fleet, “Video diffusion models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 8633–8646, 2022.
- [67] H. Aghakhani, F. Gritti, F. Mecca, M. Lindorfer, S. Ortolani, D. Balzarotti, G. Vigna, and C. Kruegel, “When malware is packin’heat; limits of machine learning classifiers based on static analysis features,” in *Network and Distributed Systems Security (NDSS) Symposium 2020*, 2020.
- [68] Ö. A. Aslan and R. Samet, “A comprehensive review on malware detection approaches,” *IEEE Access*, vol. 8, pp. 6249–6271, 2020.
- [69] A. Odena, “Semi-supervised learning with generative adversarial networks,” *arXiv preprint arXiv:1606.01583*, 2016.